

HECKS · BROWNFIELD

# Bubble context

A clean model for one slice of the business, wrapped around the systems a company already runs, and grown until the old ones can go.



## WHAT THIS COVERS

### Using Hecks with legacy systems

#### The idea

What a bubble is, and why it is a chapter and not a new construct.

#### The parts

Bluebook, hecksagon and world, and who supplies each.

#### The path

Five steps from watching legacy to retiring it.

#### The plumbing

Prewritten adapters that are thin wrappers over shared clients.

#### The help

Importer, shadow mode and a drift report, beyond adapters.

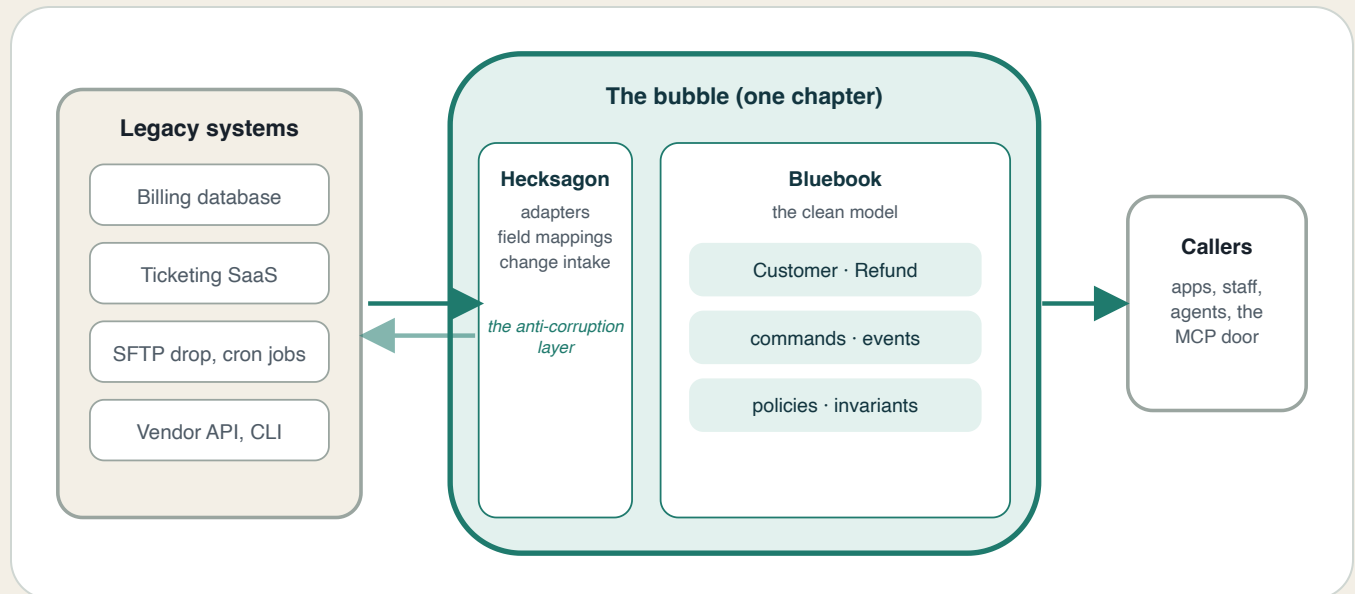
#### The decisions

What is settled and what still needs an answer.

## THE IDEA

# A bubble is a chapter wrapped around legacy

Evans called it a bubble context: a small, clean bounded context built inside a legacy system, protected by an anti-corruption layer. In Hecks it needs no new keyword. It is a chapter, loaded into a host like any other, whose hexsagon binds to the legacy systems.



- **Legacy keeps working.** Nothing in it has to change for the first step. The bubble sits beside it.
- **The model stays clean.** Legacy field names, table shapes and vendor quirks stop at the hexsagon and never reach the bluebook.
- **Callers move one at a time.** Each legacy entry point is repointed at the bubble's commands when it is ready, which is the strangler pattern.
- **Several bubbles, one host.** Refunds, Onboarding and Inventory are separate chapters that talk through **translates**, **uses\_framework** and cross-domain policies.

## THE PARTS

# Given a bluebook, supply adapters and settings

Building an integration should mean writing no application code. A company takes a bluebook (written once, shared) and supplies two things of its own.

| PART             | HOLDS                                                                                                                                                                                        | WHO SUPPLIES IT              |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| <b>Bluebook</b>  | The clean model and its ports: the aggregates, commands, events and the outside services it needs, named as ports. Says nothing about any one company.                                       | Shared, vendored, versioned  |
| <b>Hecksagon</b> | Which adapter fulfils each port, and the field mappings from legacy shapes to the bluebook's language. Inbound adapters that watch legacy live here too.                                     | The company                  |
| <b>World</b>     | Per-environment settings: base URLs, tenant ids, credential references (never values), rate limits, and the read-only switch. Dev, staging and prod are different worlds over one hecksagon. | The company, per environment |

**Check before boot.** The model check refuses a hecksagon that leaves a port unbound or binds an adapter to a port it does not implement, and refuses a world that lacks a setting an adapter declares as required.

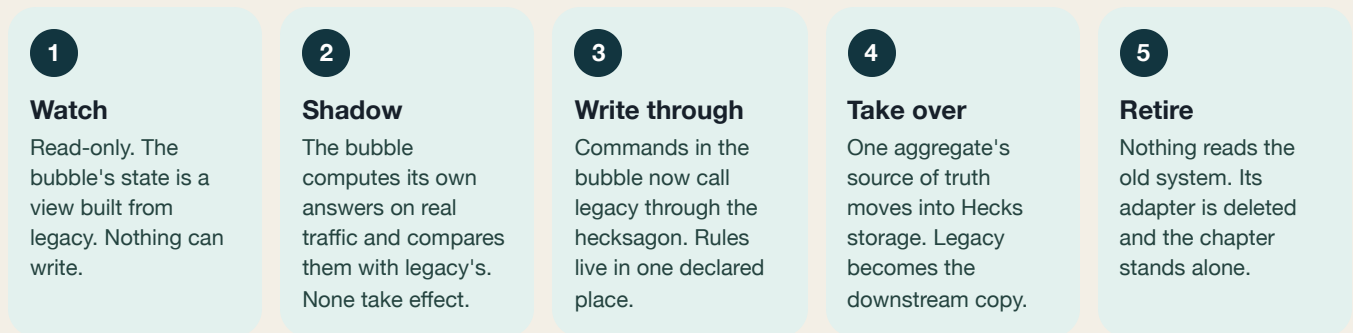
## The words, and the industry terms they stand for

| IN HECKS                                                 | INDUSTRY TERM                  | WHERE IT COMES FROM             |
|----------------------------------------------------------|--------------------------------|---------------------------------|
| <b>A bubble</b> (a chapter with legacy-facing hecksagon) | Bubble context                 | Evans, Domain-Driven Design     |
| The hecksagon's legacy-facing adapters and mappings      | Anti-corruption layer          | Evans, Domain-Driven Design     |
| Inbound adapters that only watch                         | Wire tap, change data capture  | Enterprise Integration Patterns |
| Moving callers across a step at a time                   | Strangler fig                  | Fowler                          |
| Port, adapter                                            | Ports and adapters (hexagonal) | Cockburn                        |

Only **bubble** is a new word, and it is used in prose. In code and files there is nothing new: it is a bluebook, a hecksagon and a world.

# Five steps from watching to retiring

A company can stop at any step and still have something useful. Each step adds trust before it adds risk.



## What the company gets along the way

- One **vocabulary** over systems that each name things differently.
- One **audit trail** across systems that had none.
- Swappable vendors.** Moving from one payments or CRM provider to another is an adapter change.
- Safe agent access.** An agent works through declared commands and never touches the legacy database.
- Rules in one place,** in place of stored procedures, spreadsheets and cron scripts.

## Where it gets hard

- Dirty data.** Rows that fit no aggregate need a reviewable quarantine with a reason, not an error.
- Two writers.** While both systems accept changes, a cutover needs a written procedure. The transactional outbox gives retries an owner.
- Systems that change on their own.** A legacy app that writes straight to its database needs polling, change capture or webhooks, or the bubble drifts.
- Modeling discipline.** A bluebook that mirrors the legacy schema gains nothing.

**Read-only is a setting, not a promise.** Step 1 depends on the adapter client refusing writes when the world says read-only, and on the model check confirming that a read-only adapter exposes no write operations.

# Prewritten adapters, as thin wrappers

Each adapter calls a shared client and adds only what is particular to its service. The per-service part is data, so it is written once and read by both the Ruby and Rust runtimes.

## Shared transport clients

-  **Http.** Base URL, auth, retries, rate limits, pagination, error mapping.
-  **Cli.** Argv, environment, exit codes, output parsing. Builds on the existing Shell adapter.
-  **Protocol.** SQL, SFTP and files, SMTP, SOAP, queues and webhooks. These are the workhorses of brownfield, since legacy mostly speaks them.

## Per service

-  A declarative mapping from a port ask to a request: method, path, body, response fields, auth scheme.
-  Credentials by reference, resolved at run time and never on disk.
-  A recorded-response fixture checked against the conformance corpus in both runtimes, so drift fails the build.
-  An escape hatch for services that do not fit the table, flagged as a known parity gap until ported.

| PORT                             | SERVICES                                                                                           |
|----------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Payments</b>                  | Stripe, PayPal, Square, Adyen, Paddle                                                              |
| <b>Email, SMS</b>                | Resend, SendGrid, Postmark, Mailgun, SES, SMTP/IMAP; Twilio, Vonage, MessageBird                   |
| <b>Chat, alerts</b>              | Slack, Discord, Teams, Telegram, PagerDuty                                                         |
| <b>Source, CI</b>                | GitHub, GitLab, Bitbucket, CircleCI, gh and glab                                                   |
| <b>Tickets, work</b>             | Linear, Jira, Asana, Trello, Notion, Shortcut, ServiceNow                                          |
| <b>CRM, support</b>              | Salesforce, HubSpot, Zendesk, Intercom, Pipedrive, Dynamics                                        |
| <b>Accounting, ERP, HR</b>       | QuickBooks, Xero, NetSuite, SAP, Oracle, Workday; Shopify, WooCommerce                             |
| <b>Files, workspace</b>          | S3, GCS, Azure Blob, Dropbox, Drive, Box, SFTP, CSV; Gmail, Calendar, Sheets, Microsoft Graph      |
| <b>Identity</b>                  | Auth0, Okta, Clerk, OAuth and OIDC, LDAP and Active Directory                                      |
| <b>Cloud, CLI</b>                | aws, gcloud, az, kubectl, terraform, docker, vercel, fly, Cloudflare                               |
| <b>Data</b>                      | Postgres, MySQL, MongoDB, Redis, Airtable, Supabase, Firebase, generic SQL; Snowflake, BigQuery    |
| <b>Events</b>                    | Kafka, RabbitMQ, SQS and SNS, inbound and outbound webhooks                                        |
| <b>Observe, search, AI, maps</b> | Datadog, Sentry, Grafana, Umami, PostHog; Algolia, Elastic; Anthropic, OpenAI; Google Maps, Mapbox |
| <b>Legacy protocols</b>          | SOAP and WSDL, EDI                                                                                 |

# Beyond adapters, and what is open

## More ways to help a company move

- **Importer.** Point it at a schema, an OpenAPI spec, a CLI's help or a CSV export and it drafts a chapter and hexsagon for a person to edit. This removes the blank page.
- **Shadow mode.** Real traffic, compared answers, no effect. A mismatch report is a plain go or no-go signal.
- **Drift report.** What the bubble holds against what legacy says, on demand.
- **Characterization capture.** Record real inputs and outputs from legacy and turn them into conformance fixtures.
- **Rules extraction.** Surface candidate invariants from stored procedures and scripts for a person to approve.
- **Reference bubbles.** Prebuilt chapters for common domains, so a company writes only its hexsagon and world.
- **Cutover playbook.** Generated per bubble, with the checks and rollback for each step.

## Build order

- 1. Port declarations, adapter-to-port checks and typed world settings in the model check.
- 2. Transport clients and the data-driven adapter format, Ruby and Rust.
- 3. First slice: Stripe, Slack, gh, generic SQL and webhooks, with fixtures.
- 4. The rest of the catalog.
- 5. Importer, then shadow mode and the drift report.

## Settled

- A bubble is a chapter plus its hexsagon, not a new construct.
- The model check guards the generated wiring.
- Adapters are thin wrappers over shared clients.

**Open.** Whether the field mappings live in the hexsagon (the recommendation, so the hexsagon is literally the translation layer) or in the world. Whether each chapter carries its own hexsagon (recommended) or the host has one. How read-only is enforced and checked. Whether the catalog and transport layer keep these working names.